

CS231 (Section A)

Regular Quiz 1 (Marks: 8 x 1.25 = 10)

Name:.....

Roll Number:.....

1. `fib.c` defines a recursive `fib()` function, and `main.c` has `main()` which calls `fib()` from `fib.c` and `printf()`s the result. The code is compiled as ``gcc fib.c main.c`` to generate `a.out`. What type of linking resolves the call to `fib()` versus the call to `printf()`?

- A. `fib()` is dynamically linked; `printf()` is statically linked
- B. `fib()` is statically linked; `printf()` is dynamically linked
- C. Both `fib()` and `printf()` are statically linked
- D. Both `fib()` and `printf()` are dynamically linked

2. Segments in an ELF file can be of Type "R" (Read), "RW" (Read/Write), or "RE" (Read/Executable). Which option correctly gives the segment Type containing each of these sections: `".text"`, `".rodata"`, `".data"`, `".bss"`?

- A. `.text=R`, `.rodata=RE`, `.data=RW`, `.bss=RW`
- B. `.text=RE`, `.rodata=RW`, `.data=R`, `.bss=RW`
- C. `.text=RE`, `.rodata=RE`, `.data=RW`, `.bss=RW`
- D. `.text=RE`, `.rodata=R`, `.data=RW`, `.bss=RW`

3. The ELF header stores fields such as `e_shoff` (offset to the section header table), `e_phoff` (offset to the program header table), `e_shentsize` (size of each section header entry), `e_shnum` (total number of section headers), and `e_shstrndx` (index of the section header entry describing the section header string table). Which formula correctly computes the byte offset (from the start of the file) to the section header table entry describing the section header string table (`.shstrtab`)?

- A. `e_shoff + (e_shentsize * e_shstrndx)`
- B. `e_shoff + (e_shentsize * e_shnum)`
- C. `e_phoff + (e_shentsize * e_shstrndx)`
- D. `e_shoff + e_shstrndx`

4. In the standard IA32 stack frame taught in lecture slides, `void A()` contains no code other than a call to `void B()`. `B()` contains no code other than its return statement, which it has yet to execute. The current stack frame slot for `B` is numbered 100. What stack slot will hold `A`'s Old EBP?

- A. Slot 101
- B. Slot 103
- C. Slot 102
- D. Slot 104

5. In the standard IA32 stack frame taught in lecture slides, a function `foo(int x)` is called. Right after `foo`'s prologue, `EBP` holds the address of the `foo`'s caller's `EBP` value (each stack slot is 4 bytes wide). Using pointer arithmetic on `EBP`, what offset from current `EBP` location holds the return address for this call to `foo()`?

- A. `EBP + 8`
- B. `EBP + 4`
- C. `EBP - 4`
- D. `EBP + 12`

6. Consider `void bar(int x, int y) { int x1 = x, y1 = y; return; }`. In the standard IA32 stack frame taught in lecture slides, right when `bar()` is about to execute its return statement, how many stack slots (each 4 bytes) are occupied in `bar()`'s stack frame (only)?

- A. 4 slots
- B. 5 slots
- C. 6 slots
- D. 3 slots

7. `A.o` and `B.o` are first linked together to form `AB.o`. `AB.o` is then linked with `C.o` and `D.o` to produce the final executable. `A.o` contains `.text`, `.rodata`. `B.o` contains `.text`, `.bss`. `C.o` contains `.text`, `.data`. `D.o` contains `.text`, `.rodata`, `.bss`. How many distinct sections will the final executable have?

- A. 4 sections
- B. 3 sections
- C. 5 sections
- D. 6 sections

8. `main.c` calls a function `foo()` but does not define it; `lib.c` defines `foo()`. They are compiled separately into `main.o` and `lib.o`. In `main.o`'s `.symtab`, the entry for `foo` is 'undefined' (no address yet); in `lib.o`'s `.symtab`, the entry for `foo` is 'defined' with a valid address. When the linker combines `main.o` and `lib.o` into the final executable, what happens to `foo`'s entry?

- A. Both entries remain unresolved permanently, causing a runtime crash
- B. `foo()` appears twice in the executable, each with a different address
- C. The linker deletes `foo()` entirely since it is ambiguous which file owns it
- D. The undefined reference in `main.o` is resolved using `lib.o`'s definition, so `foo()` gets a valid resolved address

Answer Key

1. B. fib() is statically linked; printf() is dynamically linked
2. D. .text=RE, .rodata=R, .data=RW, .bss=RW
3. A. $e_shoff + (e_shentsize * e_shstrndx)$
4. C. Slot 102
5. B. EBP + 4
6. D. 3 slots
7. A. 4 sections
8. D. The undefined reference in main.o is resolved using lib.o's definition, so foo() gets a valid resolved address